

Sentiment Analysis - Identifying Toxic Comments

Arpita Lonakadi
arlona@iu.edu
09-29-2024

Abstract

This project develops a machine learning model to detect toxic comments, combining text preprocessing with sentiment analysis. Various models were evaluated, with XGBoost achieving the best performance. Using TF-IDF, emoji sentiment, and VADER scores, the model successfully identifies toxic content, highlighting the potential for automated moderation in online platforms.

Summary

The objective of this project was to develop a machine learning model to identify toxic comments, defined as rude or disrespectful content likely to disrupt a discussion. Detecting such toxic comments is crucial for maintaining healthy conversations on online platforms.

The dataset used for this task comprised comments labeled as either toxic or non-toxic. Extensive preprocessing was applied to the text data, including cleaning the text, removing special characters, handling emojis, and computing sentiment scores using the VADER sentiment analyzer. Additionally, text vectorization was performed using the TF-IDF method, and emoji sentiment scores, along with VADER sentiment scores, were incorporated as features for the machine learning models.

Multiple machine learning models were evaluated, including Logistic Regression, Random Forest, Support Vector Machine (SVM), and XGBoost. After careful analysis, the XGBoost model was selected due to its superior performance in recall and F1-score, which are critical metrics for toxic comment detection. Hyperparameter tuning and threshold adjustment were conducted to improve recall, particularly for toxic comments.

The final XGBoost model achieved an accuracy of 64%, precision of 56.48%, recall of 70.05%, and an F1-score of 51.36%. Additionally, the model's ROC-AUC score of 0.81 indicated a good balance between true positive and false positive rates. The predictions generated by the model were saved in a CSV file for submission. This project highlights the potential of combining traditional text preprocessing techniques with machine learning for toxic comment detection. Future work could explore deep learning techniques to further enhance detection accuracy.

1. Introduction and Background

The goal of this project is to classify social media comments as toxic or non-toxic, utilizing machine learning techniques. Toxic comments are defined as rude, disrespectful, or unreasonable comments likely to drive users away from discussions. This type of classification is crucial for online platforms like Reddit, Twitter, and YouTube, where toxic content can harm user experience and foster negative environments. The task involves creating and evaluating models to automate the detection of toxicity based on a dataset of social media comments.

Given the complexity of language and the diverse nature of online interactions, identifying toxic comments requires effective text processing and classification techniques. Preprocessing steps such as removing special characters, user mentions, URLs, and punctuation, along with stemming and stopwords removal, help clean and standardize the text. Additionally, emoji sentiment and VADER sentiment scores are integrated to capture emotional context, which is often expressed through emojis.

To classify the comments, several machine learning models were employed, including Logistic Regression, Random Forest, Support Vector Machine, and XGBoost. By analyzing the strengths and weaknesses of each model through precision, recall, and F1-score, the project aimed to identify the most effective model for toxicity detection. XGBoost emerged as the best-performing model, showing a balance between accuracy and identifying toxic content.

2. Dataset and Methods

The datasets used in this project consist of a collection of social media comments from various platforms such as YouTube, Twitter, and Reddit.

2.1 Dataset

The **training dataset** comprises text comments, each labeled as either toxic or non-toxic. The label was determined based on the composite score, which reflects the severity and presence of toxic language in the comment. This allowed the model to learn patterns associated with toxic behavior. The training dataset contains diverse text comments, enabling the model to generalize across multiple platforms and topics.

The **testing dataset** includes similar types of comments but does not provide the toxic or non-toxic labels. This dataset was used to evaluate the model's performance, ensuring that it can accurately predict whether a comment is toxic based on its learned parameters. Both datasets include features such as the comment text, parent comment (where applicable), and associated article or platform metadata.

Training Dataset Characteristics:

Column names : text, parent_comment, article_title, article_url, platform, platform_id, composite_toxic, dtype

- 1.Text Data:** The primary feature of the training dataset is the collection of user-generated comments. These comments are derived from various platforms such as YouTube, Reddit, and Twitter, covering diverse topics like politics, social issues, entertainment, and personal opinions.
- 2.Toxicity Label:** Each comment in the training dataset is labeled as either **toxic** or **non-toxic**. The toxicity label is determined based on a composite score derived from the comment's language, tone, and the likelihood of it being harmful or offensive. Comments are labeled 1 for toxic and 0 for non-toxic.
- 3.Varying Length:** The comments range in length from very short phrases to longer, more detailed statements. This variation impacts feature extraction methods like TF-IDF and can contribute to how effectively the model handles different comment lengths.
- 4.Sentiment and Emotion:** Sentiment and emotion play a significant role in toxicity. The dataset includes emotional cues such as anger, sarcasm, joy, and frustration, which are extracted using the **VADER sentiment analyzer** and **emoji sentiment scores**.
- 5.Diverse Topics:** The dataset contains comments on a wide range of topics, which ensures that the model is exposed to different kinds of discourse. This diversity is crucial for building a generalized model capable of predicting toxicity in comments from varied contexts.

Testing Dataset Characteristics:

Column names : text, parent_comment, article_title, article_url, platform, platform_id

- 1.Text Data Without Labels:** Like the training dataset, the testing dataset contains user-generated comments, but it lacks toxicity labels. The purpose of this dataset is to evaluate the model's performance by predicting toxicity on unseen data.
- 2.Platform Metadata:** The dataset contains platform_id for each comment, which associates the comment with a specific platform or source, providing contextual information that can influence the interpretation of the comment.
- 3.Sentiment and Emotion:** Although the testing dataset does not come pre-labeled with toxicity, it includes sentiment and emoji features that can be used by the model to predict toxic behavior.
- 4.Varied Comment Length and Structure:** Similar to the training set, the comments in the testing dataset vary in length and complexity, ensuring that the model's ability to generalize across different comment structures is rigorously tested.

The analysis and prediction was done using 7 steps namely :

- 1. Data Loading:** Loaded the training and testing datasets containing user comments, toxicity labels (for training), and platform metadata for analysis.
- 2. Data Preprocessing:** Cleaned the data by removing punctuation, usernames, URLs, and stopwords, performed stemming, and calculated sentiment scores using VADER and emoji sentiment analysis.
- 3. EDA on Cleaned Data:** Conducted exploratory data analysis (EDA) on the cleaned text to examine word frequency distributions, class imbalance, and feature correlations.

4. Feature Extraction: Applied TF-IDF to transform the text into numerical features and combined them with sentiment and emoji scores for predictive modeling.
5. Model Build and Evaluation: Trained several machine learning models, including XGBoost, Logistic Regression, Random Forest, and SVM, and evaluated their performance using metrics like accuracy, precision, recall, F1-score, and confusion matrix.
6. Model Selection and Tuning: Selected XGBoost as the best model based on performance and fine-tuned hyperparameters through GridSearchCV to improve model recall for toxic comments.
7. Prediction on Test Dataset: Used the trained XGBoost model to predict toxicity on the test dataset and saved the results in the required submission format.

2.2 Data processing

2.2.1 : Data processing steps :

The data processing pipeline involved multiple steps to clean and preprocess the raw text data, extract useful features, and prepare it for model training. These steps included text cleaning, emoji handling, stopword removal, stemming, and adding sentiment scores from both emojis and the VADER sentiment analyzer.

1. Lowercasing : The text data was first converted to lowercase to ensure uniformity, as text processing is case-sensitive and could result in inconsistencies if not handled properly.
2. Removing User Mentions : User mentions in the form of `@username` were removed, as they do not contribute to the sentiment or toxicity of the comment and could introduce noise into the model.
3. Removing URLs : Any URLs present in the text were removed using regular expressions. URLs are typically irrelevant to the sentiment analysis and may introduce unnecessary noise into the model.
4. Emoji Handling : Emojis were an important aspect of the sentiment analysis, so they were converted into their text descriptions using the `emoji` library. For example, 🤔 was transformed into `face with tears of joy`. This allowed the model to capture emotional expressions conveyed through emojis. Additionally, an emoji sentiment score was calculated for each comment using a predefined dictionary. Positive emojis (e.g., 😊) were assigned a score of +1, while negative emojis (e.g., 😡) were assigned a score of -1. This emoji sentiment score was later used as a feature in the model.

5. Removing Punctuation and Special Characters : All punctuation marks and special characters were removed from the text. This helped to focus on the words and reduce the dimensionality of the text data.
6. Stopword Removal : Stopwords such as "the", "is", "in", etc., which do not carry significant meaning in sentiment analysis, were removed using NLTK's predefined list of English stopwords.
7. Stemming : The Porter Stemmer from NLTK was applied to reduce words to their root forms. For example, "running" became "run". This helped in grouping different forms of the same word and improving model performance by reducing dimensionality.
8. VADER Sentiment Scores : The VADER sentiment analyzer from the `vaderSentiment` library was used to calculate the overall sentiment of each comment. VADER is a rule-based model that is particularly good at analyzing social media text. The compound score from VADER, which is a single score that measures the overall sentiment, was added as a feature to the dataset.
9. Combining Features : After preprocessing the text, two additional features were included in the model:
 - The emoji sentiment score : A numeric feature capturing the positive or negative sentiment of the emojis used in each comment.
 - The VADER sentiment score : A numeric feature capturing the sentiment of the comment based on the text using the VADER analyzer.
10. Final Cleaned Text : After processing, the cleaned and preprocessed text, along with the emoji sentiment score and the VADER sentiment score, were used as features for model training. The processed text was vectorized using the TF-IDF vectorizer to create numerical representations for model input.

2.2.2 Exploratory Data Analysis on cleaned data

EDA was crucial for understanding the data and selecting the right features for model development. Several tests were conducted to analyze various aspects of the dataset, which informed the decision-making process for model building.

1. Class Distribution Analysis:

A count plot was used to examine the distribution of toxic and non-toxic comments. The plot showed a significant class imbalance, with far more non-toxic comments than toxic ones. This imbalance indicated the potential need for techniques such as class weighting or threshold tuning to improve recall for toxic comments. It also suggested that precision could be easily achieved for non-toxic comments, while recall would be more challenging for the toxic ones.

2. Word Cloud Analysis:

Word clouds were generated to visualize the most frequent words in toxic and non-toxic comments. Toxic comments contained aggressive and offensive language (e.g., "fuck," "idiot"), while non-toxic comments used more neutral or general terms. This analysis helped confirm that linguistic features, especially the frequency of negative language, would play a significant role in toxicity detection.

3. Emoji Sentiment Distribution:

The distribution of emoji sentiment scores was examined to assess the impact of emojis on comment toxicity. Negative emojis were more common in toxic comments, while positive or neutral emojis appeared more frequently in non-toxic ones. This highlighted the importance of including emoji sentiment scores as features, as they provided an additional emotional dimension to the comments.

4. VADER Sentiment Score Analysis:

VADER sentiment scores, which measure the emotional tone of the comments, were analyzed next. Toxic comments generally had more negative VADER scores, confirming that sentiment analysis could enhance the detection of toxicity. The VADER sentiment scores were therefore included as part of the feature set to complement text-based features.

5. Correlation Heatmap:

A correlation heatmap was used to analyze relationships between numerical features like ``emoji_score``, ``vader_score``, and ``toxic_label``. The heatmap showed a moderate correlation between VADER scores and toxicity, confirming the relevance of this feature. The emoji scores had a weaker correlation but still provided value in the overall feature set.

Combined, these EDA steps provided critical insights that guided feature engineering and model selection, ensuring that both linguistic and sentiment-based features were effectively captured.

Logistic Regression and XGBoost were preferred due to the EDA results, which showed strong linear separability between toxic and non-toxic comments, as seen in sentiment and emoji scores. Logistic Regression handled these linear relationships well, while XGBoost captured complex interactions. Feature extraction, driven by EDA insights, included sentiment and emoji scores to complement text-based features.

2.3 ML methods

2.3.1 : Feature extraction :

In this project, feature extraction was crucial for transforming raw text data into a format suitable for machine learning models. Two main techniques were used:

1. **TF-IDF (Term Frequency-Inverse Document Frequency)**: TF-IDF was applied to convert the cleaned text into numerical features. It measures the importance of words in the document relative to the entire dataset. Words frequently occurring in a specific comment but

rarely elsewhere are assigned higher weights, helping models focus on contextually important terms.

2.Sentiment Features (VADER and Emoji Sentiment Scores): Along with text, emotional content was also captured through VADER sentiment scores and emoji sentiment scores. VADER computes the overall sentiment of a comment (ranging from negative to positive), while the emoji sentiment score assigns a numerical value (+1 for positive emojis, -1 for toxic emojis). These additional features enriched the dataset by providing insights into the emotional tone of comments.

These extracted features (TF-IDF vectors, VADER, and emoji sentiment scores) were combined and used to train models. By capturing both text and emotional aspects, this approach improved model performance, especially in identifying toxic comments with emotional undertones.

2.3.2 : Model build

In this project, several machine learning methods were implemented for text classification, specifically Logistic Regression, Random Forest, Support Vector Machine (SVM), and XGBoost. Each model has unique strengths and was applied to predict the toxicity of comments. Here's a brief overview:

1. Logistic Regression: A simple yet effective linear classifier used for binary classification. It estimates the probability of a given class using the logistic function, ideal for separating linearly distinguishable data.
2. Random Forest: An ensemble method that combines multiple decision trees to improve accuracy. Each tree in the forest provides a class prediction, and the majority vote determines the final output, enhancing generalization and reducing overfitting.
3. Support Vector Machine (SVM): A robust method that aims to find a hyperplane that best separates the classes by maximizing the margin between data points from different categories. It works well for high-dimensional spaces and is less prone to overfitting.
4. XGBoost: An advanced gradient boosting algorithm, XGBoost builds models sequentially, with each new model correcting errors from the previous one. It's highly efficient, accurate, and handles non-linear relationships between features well.

Each model was trained on TF-IDF features and additional sentiment scores (VADER and emoji sentiment) to better capture both text-based and emotional patterns in the data.

3. Evaluations and Findings

3.1 Model selection

In this project, multiple models were trained and evaluated using key classification metrics: Accuracy, Precision, Recall, and F1-Score. These metrics provide insights into how well the models predict toxic and non-toxic comments.

1. Accuracy measures the overall percentage of correct predictions, but it may not always reflect performance on imbalanced datasets like ours.

2. Precision indicates how many of the predicted toxic comments were actually toxic. A high precision means fewer false positives.

3. Recall measures how many actual toxic comments were correctly identified by the model. A high recall means fewer false negatives, which is important when the goal is to capture as many toxic comments as possible.

4. F1-Score provides a balance between precision and recall, making it a more reliable measure when dealing with imbalanced datasets like this one.

Logistic Regression:

- Accuracy: 76.38%
- Precision: 69.09%
- Recall: 18.10%
- F1-Score: 28.68%

Logistic regression showed reasonable accuracy and precision but had a low recall, indicating that it missed many toxic comments.

Random Forest:

- Accuracy: 75.75%
- Precision: 72.22%
- Recall: 12.38%
- F1-Score: 21.14%

While Random Forest had high precision, its recall was even lower than logistic regression, meaning it missed a significant number of toxic comments.

Support Vector Machine (SVC):

- Accuracy: 75.62%
- Precision: 65.96%
- Recall: 14.76%
- F1-Score: 24.12%

SVC showed similar performance to Random Forest, with decent precision but a poor recall.

XGBoost:

- Accuracy: 75.50%
- Precision: 56.48%
- Recall: 29.05%
- F1-Score: 38.36%

XGBoost, despite having the lowest precision, achieved the highest recall and F1-Score, indicating a better balance between precision and recall. This model was able to capture more toxic comments compared to the others, making it the most effective for this task.

Key Findings:

- XGBoost showed the best performance in terms of Recall and F1-Score, making it the preferred model for detecting toxic comments.
- Logistic Regression performed well in precision but struggled with recall, which could lead to missing many toxic comments.
- Both Random Forest and SVC struggled to balance precision and recall, performing less optimally than XGBoost.

In summary, XGBoost was selected as the best model due to its ability to strike a better balance between precision and recall, which is crucial in a task like toxic comment classification where identifying toxic comments is of high priority.

3.2 Model tuning

After evaluating several models, the XGBoost model showed the best balance between precision and recall, but there was room for improvement. Model tuning, also known as hyperparameter optimization, was performed on XGBoost to enhance its performance. The process involved adjusting key hyperparameters that affect the model's complexity, learning rate, and tree construction. The key hyperparameters considered were:

1. `n_estimators`: This controls the number of trees used in the model. More trees can improve model accuracy but also increase training time and the risk of overfitting.
2. `learning_rate`: This defines the step size at each iteration while moving toward the optimal set of parameters. A lower learning rate may lead to better results, but it requires more trees (higher `n_estimators`).
3. `max_depth`: This defines the maximum depth of each tree. Deeper trees can model more complex patterns but are more prone to overfitting.
4. `colsample_bytree`: This is the fraction of features to be randomly selected for each tree. Lower values can help reduce overfitting.
5. `subsample`: This is the fraction of samples used to grow each tree. A lower value can help reduce overfitting by introducing randomness into the model.

Grid Search for Hyperparameter Tuning:

To identify the optimal combination of hyperparameters, a grid search was performed using cross-validation. The grid search tested different values for each hyperparameter, evaluating the performance of each combination based on the F1-Score. This metric was selected to strike a balance between precision and recall.

Best Hyperparameters:

After the grid search, the following optimal values were found:

- n_estimators: 300
- learning_rate: 0.2
- max_depth: 5
- colsample_bytree: 1.0
- subsample: 0.7

These tuned hyperparameters improved the model's ability to correctly identify toxic comments without significantly sacrificing precision.

Threshold Adjustment:

In addition to hyperparameter tuning, the decision threshold was also adjusted. The default threshold for XGBoost is 0.5, but by lowering it to 0.3, the recall improved significantly, capturing more toxic comments at the cost of a slight drop in precision. This threshold tuning ensured a better balance in the final model, which is crucial when false negatives (missing toxic comments) are more concerning than false positives.

Conclusion:

By tuning both the hyperparameters and the decision threshold, the performance of the XGBoost model improved, making it the most suitable model for detecting toxic comments in this dataset.

3.3 Results

The project aimed to classify toxic comments using text preprocessing, emoji sentiment, and VADER sentiment analysis. Logistic Regression, Random Forest, SVM, and XGBoost models were evaluated. XGBoost performed best, achieving an accuracy of 64.50%, recall of 70.05%, and an F1-Score of 51.36% after tuning.

3.4 References :

1. <https://github.com/PatilMrudu/Youtube-Comments-Sentimental-Analysis/blob/main/README.md>
2. <https://www.geeksforgeeks.org/sentiment-analysis-of-youtube-comments/>
3. <https://www.analyticsvidhya.com/blog/2021/06/twitter-sentiment-analysis-a-nlp-use-case-for-beginners/>